School of Computing and Information Technology
University of Wollongong

Session: **Spring 2025**
Lecturer: **Janusz R. Getta**

ISIT912 Big Data Management
Assignment 1
Published on 28 July 2025

Scope

This assignment includes the tasks related to implementation of HDFS applications, implementation of MapReduce applications and design of a complex MapReduce application.

This assignment is due on **Saturday, 23 August 2025, 7:00pm** (sharp).

This assignment contributes to **10%** of the total evaluation in the subject.

The assignment consists of 5 tasks and specification of each task starts from a new page.

Only electronic submission through Moodle at:
`https://moodle.uowplatform.edu.au/login/index.php`
will be accepted. A submission procedure is explained at the end of Assignment 1 specification.

A policy regarding late submissions is included in the subject outline.

Only one submission of Assignment 1 is allowed and only one submission per student is accepted.

A submission marked by Moodle as "late" is always treated as a late submission no matter how many seconds it is late.

A submission that contains an incorrect file attached is treated as a correct submission with all consequences coming from the evaluation of the file attached.

All files left on Moodle in a state "`Draft(not submitted)`" will not be evaluated.

A submission of compressed files (zipped, gzipped, rared, tared, 7-zipped, lhzed, … etc) is not allowed. The compressed files will not be evaluated.

An implementation that does not compile well due to one or more syntactical and/or run time errors scores no marks.

Using any sort of Generative Artificial Intelligence (GenAI) for this assignment is NOT allowed !

It is expected that all tasks included within **Assignment 1** will be solved **individually without any cooperation** with the other students. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or office hours. Plagiarism will result in a **FAIL** grade being recorded for the assessment task.

Task 1 (1 mark)
Command line interface to HDFS

Use a text editor to create three text files that consists of minimum 4 lines. Insert into the first line a name of file. Insert into the second line your full name and your student number. Insert into the third line day and time of a laboratory class you are enrolled in. Insert into the fourth line any text longer than 20 characters. The names of files are up to you.

(1) Open `Terminal` window and navigate to a folder where the text files are stored. Use `cat` command to list both files in `Terminal` window.

Use a command line interface to HDFS to perform the following operations in `Terminal` window opened in the previous step. Note, that implementation of the steps listed below may need more than one command per single step.

(2) Create three folders in a home folder of HDFS files system. The names of folders are up to you.

(3) Upload three files created at the beginning to the folders created in step (2). Each file must be located in a different folder.

(4) List the contents of the folders created in a step (2).

(5) List the contents of the files uploaded to HDFS.

(6) Merge the files located in HDFS into one file located in your home folder in HDFS. An order in which the files are merged is up to you. A name of merged file is also up to you. The files must be merged in HDFS. It is not allowed to copy the files into a local file system and merge the files there.

(7) List the contents of the file merged in HDFS.

(8) Copy the merged file into a local file system.

(9) Remove all three files created in HDFS.

(10) Remove the folders created in a step (2).

When ready, copy into a clipboard the contents of `Terminal` window with the operations processed above and the results listed in the window and Paste the results from a clipboard into a text file `solution1.txt`.

Deliverables
A file `solution1.txt` that contains a listing of operations performed above and the results of operations. A file `solution1.txt` must be created through Copy/Paste of

the contents of `Terminal` window into a file `solution1.txt`. No screen dumps are allowed and no screen dumps will be evaluated.

Task 2 (2 marks)
Python interface to HDFS

Implement the steps (1), (2) and (3) from Task 1. No report is expected from the implementation of the steps (1), (2) and (3) from Task 1.

Use `Snakebite` client library to implement in Python an application that performs the following operations.

(1) List the names of files and folders included in a home folder in HDFS.

(2) List the contents of the folders implemented in a step (2) of Task 1.

(3) List the contents of the files included in the folders in step (3) of Task 1.

When ready open `Terminal` Window and navigate to a folder where your Python application is located. Next, use `cat` command to list a file with your Python application.

Next, process your Python application in `Terminal` window. Finally, copy into a clipboard the contents of `Terminal` window and paste the contents of a clipboard into a file `solution2.txt`.

You can find more information about `Snakebite` client library in the lecture slides, laboratory specifications and at
`https://snakebite.readthedocs.io/en/latest/client.html`.

Deliverables
A file `solution2.txt` that contains a listing of Python application that performs the operations listed above together with the results of operations. A file `solution2.txt` must be created through Copy/Paste of the contents of `Terminal` window into a file `solution2.txt`. No screen dumps are allowed and no screen dumps will be evaluated.

Task 3 (3 marks)
MapReduce streaming with Python

Implement in Python a Hadoop streaming application that has the functionality equivalent to the functionality of the following SQL statement:

```
SELECT student_no, score1 + score2 + score3
FROM assignment.txt
WHERE (score1+score2+score3)/3 > given_value;
```

Assume that all lines in an input data set `assignmemnt.txt` are consistent with the following format.

```
student_no score1 score2 score3
```

For example:

```
7695464 10 20 30
3456781 0 15 10
6745234 10 20 30
6578345 0 0 0
6435679 7 2 10
6834455 2 20 30
5648121 2 5 30
```

The contents of a file `assignmenmts.txt` is up to you as long as it is consistent with a format explained above.

A value of `given-value` is up to you, however must be selected such that at least one line is listed by the application.

A value of `given-value` must be passed through a parameter of your program.

Save your solution in a file `solution3.py`.

When ready perform the following actions.

(1) Open `Terminal` window and navigate to a folder where a file `assignments.txt` is stored.

(2) Use `cat` command to list the contents of a file `assignmengts.txt` in `Terminal` window.

(3) Upload a file `assignments.txt` to HDFS. Location of the file in HDFS is up to you.

(4) Navigate to a folder where a file `solution3.py` is stored.

(5) Use `cat` command to list the contents of a file `solution3.py` in `Terminal` window.
(6) Use `chmod` command to change user's access permission to a file `solution3.py`.

(7) Process a program in `solution3.py` with the Hadoop streaming feature.

(8) Display the results stored by your program in HDFS.

Finally, copy into a clipboard the contents of `Terminal` window and paste the contents of a clipboard. into a file `solution3.txt`.

Deliverables
A file `solution3.txt` that contains a listing of operations listed above and the outcomes from the operations. A file `solution3.txt` must be created through Copy/Paste of the contents of `Terminal` window into a file `solution3.txt`. No screen dumps are allowed and no screen dumps will be evaluated.

Task 4 (4 marks)
MapReduce streaming with Python

Implement in Python a Hadoop streaming application that has the functionality equivalent to the functionality of the following SQL statement.

```
SELECT location, AVG(temperature)
FROM measurement.txt
GROUP BY location
```

All lines in an input data set `measurement.txt` are consistent with the following format.

```
location day month year temperature.
```

For example:

```
Dapto 2 12 2005 30
Wollongong 2 12 2005 30
Dapto 2 12 2005 30
Sydney 2 12 2005 30
Brisbane 2 12 2006 30
Dapto 2 2 2005 35
Dapto 1 8 2006 20
Adelaide 3 12 2006 35
Brisbane 2 10 2007 25
Dapto 2 7 2006 15
```

The contents of a file `measurement.txt` is up to you as long as it is consistent with a format explained above.

Save your solution in the files `solution4m.py` and `solution4r.py`.

When ready perform the following actions.

(1) Open `Terminal` window and navigate to a folder where a file `measurement.txt` is stored.

(2) Use `cat` command to list the contents of a file `measurement.txt` in `Terminal` window.

(3) Upload a file `measurement.txt` to HDFS. Location of the file in HDFS is up to you.

(4) Navigate to a folder where the files `solution4m.py` and `solution4r.py` are stored.

(5) Use `cat` command to list the contents of the files `solution4m.py` and `reducer4.py` in `Terminal` window.

(6) Use `chmod` command to change user's access permission to the files `mapper4.py` and `solution4r.py`.

(7) Process your application with the Hadoop streaming feature.

(8) Display the results stored in HDFS by your application.

Finally, copy into a clipboard the contents of `Terminal` window and paste the contents of a clipboard. into a file `solution4.txt`.

Deliverables

A file `solution4.txt` that contains a listing of operations listed above and the outcomes from the operations. A file `solution4.txt` must be created through Copy/Paste of the contents of `Terminal` window into a file `solution4.txt`. No screen dumps are allowed and no screen dumps will be evaluated.

Task 5 (5 marks)
Describing MapReduce implementation

The files `region_a.txt`, `region_b.txt` and `region_c.txt` contain information about the application processing time submitted by each customer in three different regions.

All lines in all three files are consistent with the following structure.

```
customer# total_time
```

For example:

```
region_a.txt           region_b.txt           region_c.txt
customer# total_time   customer# total_time   customer# total_time
0000001   25           0000002   20           0000001   25
0000008   13           0000001   10           0000007   34
0000001   45           0000010   100          0000007   25
0000008   13           0000001   10           0000007   34
0000007   45           0000010   40           0000007   28
```

Note that some customers processed their applications in only one region, some of them used two regions and some of them used all three regions.

Your task is to explain how to implement a MapReduce application that that *finds total processing time of all customers who processed their applications in all three regions*.

In the example given above only customer `0000001` processed the applications in all three regions. Its total processing time is equal to `25+45+10+10+25 = 115`.

Answer the following questions in your explanations.

(1) What are the parameters of your application ?

(2) What information is filtered out by a mapper ?

(3) What key-value pairs are created by a mapper ?

(4) How key-value pairs are created by a mapper ?

(5) What key-value pairs are processed by a reducer ?

(6) How key-value pairs are processed by a reducer ?

(7) What is the format of the final results ?

(8) How to upload data to HDFS ?

(9) How to prepare your application for processing ?

(10) How to list the final results ?

Save your explanations in a file `solution5.pdf`.

You must include the text of all questions and your answers in your solution. The marks will be deducted for missing questions and answers. The comprehensive explanations related to all stages of data processing are expected. Try to be as specific as it is possible. This task does not require you to write any code in a programming language and/or pseudocode.

Deliverables
A file `solution5.pdf` with the comprehensive explanations how would you implement a MapReduce application that *finds total processing time of all customers who processed their applications in all three regions*.

Submission of Assignment 1

Note, that you have only one submission. So, make it absolutely sure that you submit the correct files with the correct contents. No other submission is possible !

Submit the files **solution1.txt**, **solution2.txt**, **solution3.txt**, **solution4.txt** and **solution5.pdf** through Moodle in the following way:

(1) Access Moodle at **http://moodle.uowplatform.edu.au/**
(2) To login use a **Login** link located in the right upper corner the Web page or in the middle of the bottom of the Web page
(3) When logged select a site **ISIT912/312 (S225) Big Data Management**
(4) Scroll down to a section **Assessment items (Assignments)**
(5) Click at **In this place you can submit the outcomes of your work on the tasks included in Assignment 1 for ISIT912 students** link.
(6) Click at a button **Add Submission**
(7) Move a file **solution1.txt** into an area **File submissions**. You can also use a link **Add**...
(8) Repeat a step (7) for the files **solution2.txt**, **solution3.txt**, **solution4.txt** and **solution5.pdf**
(9) Click at a button **Submit assignment.**
(10) Click at the checkbox with a text attached: **By checking this box, I confirm that this submission is my own work, I accept responsibility for any copyright infringement that may occur as a result of this submission, and I acknowledge that this submission may be forwarded to a text-matching service**.
(11) Click at a button **Continue**
(12) Check if **Submission status** is **Submitted for grading**.

End of specification